408 考研四科融合思维导图（完整版）

融合说明

- **数据结构**：完整融合自「408思维导图...
- **计算机网络**：完整保留自「计算机网络...
- **计算机组成原理、操作系统**：本次新...

- 一、绪论
  - 1 基本概念
    - 数据与信息
      - 数据是信息的载体
    - 数据元素：数据的基本单位
      - 数据项：构成数据元素的不可分割的最小单位
    - 数据对象
      - 具有相同性质的数据元素的集合
      - 是数据的一个子集
    - 数据类型
      - 原子类型：其值不可再分
      - 结构类型：其值可以分解为若干成分
      - 抽象数据类型（ADT）：不关心存储结构
    - 数据结构
      - 结构 = 数据元素相互之间的关系
      - 数据结构是一组存在特定关系的数据元素的集合
  - 2 数据结构三要素
    - 逻辑结构
      - 线性结构：线性表、栈&队列&串（受限线性表）、数组（推广线性表）
      - 非线性结构：树、图、集合
    - 存储结构（物理结构）
      - 顺序存储（连续）
      - 链式存储（指针）
      - 索引存储（索引表：非顺序、离散）
      - 散列存储（hash存储）
    - 数据运算
      - 运算的定义：针对逻辑结构
      - 运算的实现：针对存储结构
  - 3 算法
    - 算法的五大特征
      - 有穷性、确定性、可行性
      - 输入：零个或多个
      - 输出：一个或多个
    - 算法的目标
      - 正确性、可读性、健壮性、效率（时间与空间）
    - 算法的效率
      - 时间复杂度
      - 空间复杂度
- 二、线性表
  - 1 基本概念
    - 位序：从1开始；下标：从0开始
    - 【易错】线性表是逻辑结构，顺序表和链表是存储结构
  - 2 实现
    - 顺序表（顺序存储）
      - 特点：支持随机存取、存储密度高
      - 分配方式：静态分配、动态分配
    - 链表（链式存储）
      - 单链表
      - 双链表
      - 循环链表
        - 循环单链表
        - 循环双链表
      - 静态链表
        - 借助数组实现，需要预先分配一块连续的存储空间
        - 指针是结点的相对地址（数组下标），又称游标
  - 3 时间复杂度分析 ★
    - 按值查找
      - 无序顺序表：O(n)
      - 有序顺序表：O(log2n)
      - 链表：O(n)
    - 按序查找（按序号i）
      - 顺序表：O(1)
      - 链表：O(n)
- 三、栈和队列
  - 1 栈
    - 基本概念
      - 栈的数学性质：n个元素进栈，不同的出栈顺序个数为【卡特兰数】
    - 实现
      - 顺序栈（顺序存储）
        - 栈顶指针 S.top 的两种初始化方案
          - 初始值-1（通常）：栈空 S.top==-1；栈满 S.top==MaxSize-1；栈长 S.top+1
          - 初始值0（可选）：栈空 S.top==0；栈满 S.top==MaxSize；栈长 S.top
      - 共享栈（顺序存储）
        - 两个顺序栈共享存储空间
        - 0号栈空：top0==-1；1号栈空：top1==MaxSize
        - 栈满：top1 - top0 == 1（两栈顶指针相邻）
      - 链栈（链式存储）
        - 所有操作在单链表的表头进行
    - 应用
      - 括号匹配
      - 中缀表达式 → 后缀表达式转换
      - 后缀表达式求值：遇到数值进栈；遇到操作符出栈2个→运算→结果进栈
      - 递归算法
        - 递归转非递归通常借助栈
        - 消除尾递归通常可借助循环
  - 2 卡特兰数应用 ★
    - 公式：F(n) = C(2n, n) / (n+1)
    - 求出栈序列数：n个不同元素依次入栈，求不同出栈序列个数
    - 求二叉树形态数：n个结点能组成的二叉树的不同形态个数
      - 给定遍历序列时：一种形态+一组序列唯一确定一棵二叉树 → 转化为求形态数
    - 括号匹配序列数：n对括号的合法匹配序列个数
      - 等价于左括号入栈、右括号出栈 → 转化为出栈序列数
    - 受限的双端队列：根据条件合理使用卡特兰数推导
  - 3 队列
    - 实现
      - 循环队列（顺序存储）
        - Q.front指向队头元素，Q.rear指向队尾元素的下一个
        - 队空：Q.front == Q.rear
        - 区分队空与队满（3种方案）
          - 牺牲一个存储单元：队满 (Q.rear+1)%MaxSize == Q.front
          - 增设Q.size：==MaxSize队满，==0队空
          - 增设tag：删除导致front==rear时tag=0；插入导致front==rear时tag=1
      - 链式队列（链式存储）
        - 队空：Q.front == Q.rear（不带头结点时两者均为null）
        - 【易错】带头结点时插入删除无需判断队空；不带头结点需要特殊处理
    - 双端队列
      - 两端均可插入、删除
      - 衍生：输出受限 / 输入受限的双端队列
      - 常考：给定输入序列，判断某输出序列是否合法
      - 难点（通常不考）：用卡特兰数计算可能的输出序列个数
    - 应用
      - 层次遍历二叉树
- 四、数组与特殊矩阵
  - 1 数组的压缩存储
    - 对称矩阵
    - 三角矩阵（上三角 / 下三角）
    - 三对角矩阵：三条对角线元素按行优先存放于一维数组
    - 稀疏矩阵
      - 非零元素 → 三元组（行标，列标，值）
      - 数组存储：按某种规律（如行优先）存储三元组
      - 十字链表存储
- 五、串（模式匹配）
  - 1 串的实现
    - 定长顺序存储：静态分配定长数组；超长采取"截断"
    - 堆分配存储（顺序存储）：动态分配，存储于堆区
    - 块链存储（链式存储）：一个结点存一个块（多个字符），占不满用"#"补位
  - 2 模式匹配算法 ★★★
    - 暴力匹配（BF）
    - KMP
      - ①求模式串t的PM表：每个前缀的【最长相等前后缀长度】
      - ②PM表整体右移、首位填-1 → next数组
        - next[0]=-1，next[1]=0，j>1时next[j]≥0
        - 【易错】next数组后一项最多比前一项大1
      - ③匹配：失配（t[j]≠s[i]）时 j=next[j] 再重新比较；j==-1时 i++、j++（整体右移一位）
      - ④next可进一步优化为nextval
        - nextval[0]=-1
        - 若 t[j]==t[next[j]]，则 nextval[j]=nextval[next[j]]
        - 否则 nextval[j]=next[j]
  - 3 KMP时间复杂度
    - 暴力：最坏O(mn)，实际一般近似O(m+n)
    - KMP：O(m+n)
      - 主串与子串有很多"部分匹配"时优势明显；主要优点是主串不回溯
  - 1 树的基本概念
    - 结点的度 & 树的度（m叉树的度≤m）
    - 无序树 & 有序树
    - 路径：一定从上向下（双亲→孩子）
    - 路径长度：经过的边数
    - 树的路径长度：树根到每个结点的路径长度之和
    - 双亲表示法（顺序存储）：类似静态链表，下标+双亲下标

- 一、计算机网络体系结构
  - 1 基本概念
    - 定义：互连的、自治的计算机系统的集合
    - 组成：硬件 + 软件 + 协议
    - 工作方式
      - 边缘部分：直接使用（C/S方式、P2P方式）
      - 核心部分：为边缘提供服务（大量网络+路由器）
    - 功能：数据通信、资源共享、分布式处理、提高可靠性、负载均衡
  - 2 分类
    - 按范围：WAN > MAN > LAN > PAN
    - 按使用者：公用网 / 专用网
    - 按交换技术：电路交换 / 报文交换 / 分组交换
    - 按拓扑：总线 / 星型 / 环形 / 网状
    - 按传输技术：广播式 / 点对点式
  - 3 性能指标 ★★★
    - 速率：b/s；1kb/s = 10^3 b/s（10进制）
    - 带宽：最高数据率 或 信道频带宽度
    - 吞吐量：实际通过的数据量
    - 时延
      - 发送时延 = 分组长度 ÷ 发送速率
      - 传播时延 = 信道长度 ÷ 电磁波速率
      - 处理时延 + 排队时延（路由器）
      - 【易错】发送时延与链路长度无关；传播时延与速率无关
      - 【易错】提高带宽降低的是发送时延
    - 时延带宽积 = 传播时延 × 带宽
      - 意义：以比特为单位的链路长度
    - RTT：从发送到收到确认的总时间
    - 信道利用率：D = D0/(1−U)；利用率过高时延爆炸
  - 4 分层结构
    - 协议三要素：语法、语义、同步
    - PDU对照
      - 物理层：比特
      - 数据链路层：帧
      - 网络层：IP数据报 / 分组
      - 传输层：报文段 / 用户数据报
      - 应用层：报文
    - 三种模型对照
      - OSI：物—数—网—传—会—表—应
      - TCP/IP：网络接口—网际—传输—应用
      - 教学5层：物—数—网—传—应
      - 【考点】TCP/IP网络层：无连接不可靠；传输层：TCP可靠 + UDP不可靠
- 二、物理层
  - 1 通信基础
    - 信源 → 信道 → 信宿
    - 模拟信号（连续）/ 数字信号（离散）
    - 码元：固定时长的信号波形
      - 1码元携带 log2V bit（V为状态数）
      - eg：QAM-16 → 1波特 = 4比特
  - 2 两大定理 ★★★
    - 奈奎斯特（无噪声）
      - 极限速率 = 2W·log2V
      - 限制的是码元速率（波特率）
    - 香农（有噪声）
      - 极限速率 = W·log2(1+S/N)
      - dB = 10·lg(S/N)
      - eg：30dB → S/N = 1000
      - 限制的是信息速率（比特率）
    - 【易错】两者同时给出，取较小值
  - 3 编码与调制
    - 编码
      - 归零RZ：每个码元末尾归0，自同步
      - NRZI：电平翻转=0
      - 曼彻斯特：码元中间必跳变
        - 【易错】效率50%；10Mb/s以太网 → 20MBaud
      - 差分曼彻斯特：码元开始处跳变=0
    - 调制：ASK调幅 / FSK调频 / PSK调相 / QAM幅相结合
  - 4 传输介质
    - 导向
      - 双绞线、同轴电缆
      - 光纤
        - 多模：发光二极管，近距离
        - 单模：激光二极管，远距离
    - 非导向：无线电波（全方向）/ 微波（直线）/ 红外 / 激光
  - 5 物理层设备
    - 中继器：5-4-3规则
    - 集线器Hub
      - 多端口中继器，物理层
      - 【易错】不隔离冲突域、不隔离广播域
      - 【易错】所有站点共享带宽、半双工
- 1 功能
  - 封装成帧、差错控制、流量控制、透明传输
- 2 组帧
  - 字符计数法：首字节表示帧长；出错→灾难性
  - 字节填充法：PPP使用
  - 比特填充法：5个连续1后填0；HDLC使用
  - 违规编码法：非法编码做定界
- 3 差错控制
  - 检错
    - 奇偶校验：1bit检错，不能纠错
    - CRC：模2除法
      - 【易错】只能检错不能纠错
  - 纠错：海明码
    - ★ 2^r ≥ k + r + 1
    - eg：4位数据 → r = 3
    - 码距d：检错 d−1 位；纠错 floor((d−1)/2) 位
- 4 滑动窗口可靠传输 ★★★
  - 停等S-W：发窗口1，收窗口1
    - 信道利用率 U = TD/(TD+RTT+TA)
  - GBN
    - 发送窗口 ≤ 2^n − 1；接收窗口 = 1
    - 【易错】失序帧丢弃，不缓存
    - 失帧后重传其后所有帧
  - SR
    - 发送窗口 ≤ 2^(n−1)；接收窗口 ≤ 2^(n−1)
    - ★ 发送窗口 + 接收窗口 ≤ 2^n
    - 失序帧缓存，只重传错的
  - 【易错】n = 序号比特数，窗口值 ≠ n
- FDM：同时不同频

第四部分　计算机网络（408）

- 三、数据链路层
  - 5 介质访问控制 ★★★
    - 信道划分（静态）
      - TDM：同频不同时
      - STDM：统计时分，按需分配
      - WDM：光的频分复用
      - CDM/CDMA
        - 码片向量：S·S = 1；S·(−S) = −1；S·T = 0
        - 内积1→发1；内积-1→发0；内积0→没发
    - 随机访问（动态）
      - 纯ALOHA：想发就发；利用率18.4%
      - 时隙ALOHA：时间片开始才能发；利用率36.8%
      - 1-坚持CSMA：忙则一直监听，空闲立即发
      - 非坚持CSMA：忙则等随机时间
      - p-坚持CSMA：空闲以概率p发送
    - CSMA/CD（有线以太网，半双工）
      - 口诀：先听后发、边听边发、冲突停发、随机重发
      - 争用期 = 2τ
      - ★ 最小帧长 = 2τ × 数据率
        - 10Mb/s以太网 → 64B = 512bit
      - 退避算法
        - k = min(重传次数，10)
        - r ∈ [0, 2^k − 1] 随机
        - 退避时间 = r × 争用期
        - 16次失败 → 放弃
    - CSMA/CA（无线802.11）
      - 不能用CD原因：信号衰减快无法边发边检 + 隐藏站
      - IFS帧间间隔：SIFS最短，优先级最高
      - 随机退避 + ACK确认
      - RTS/CTS信道预约 + NAV虚拟载波监听
    - 轮询访问：令牌传递协议
  - 6 局域网与以太网 ★★★
    - 802分层：LLC子层 + MAC子层
    - 以太网服务：无连接、不可靠（只丢弃不重传）
    - MAC帧格式
      - 目的MAC(6B) + 源MAC(6B) + 类型(2B) + 数据(46~1500B) + FCS(4B)
      - 最小64B，最大1518B
      - 【易错】数据 < 46B 需填充；IP数据报 ≤ 1500B（MTU）
      - 【易错】无帧结束定界符
    - MAC地址：48位
      - 全1 = FF-FF-FF-FF-FF-FF = 广播地址
  - 7 设备对比 ★★★
    - Hub：物理层；不隔离冲突域、不隔离广播域
    - 交换机/网桥：链路层；隔离冲突域、不隔离广播域
      - 自学习：按源MAC建表，查表转发
      - 【易错】每端口独享带宽，可全双工
    - 路由器：网络层；隔离冲突域 + 隔离广播域
    - VLAN：802.1Q，逻辑隔离广播域
- 四、网络层 ★★★
  - 1 IPv4首部
    - 固定20B + 可选40B，最大60B
    - 字段单位
      - 首部长度：单位4B，最大值15 → 60B
      - 总长度：单位1B，最大65535B
      - 片偏移：单位8B
    - TTL：每过一个路由器减1，为0丢弃
    - 协议字段：TCP=6；UDP=17；ICMP=1；IGMP=2
    - 【易错】首部校验和只校验首部；逐跳重新计算
  - 2 IP分片 ★★★
    - 触发：数据报长度 > MTU
    - 每片可装数据 = floor((MTU−20)/8) × 8
    - 除最后一片，其余 MF = 1
    - 片偏移 = 该片起始字节 ÷ 8
    - 【易错】每片都有20B首部，总开销增加
    - 【易错】中间路由器不分片不重组；仅最终目的主机重组
  - 3 IP地址分类
    - A类：0开头；1~126；默认掩码255.0.0.0
    - B类：10开头；128~191；默认掩码255.255.0.0
    - C类：110开头；192~223；默认掩码255.255.255.0
    - D类：组播；E类：保留
    - 特殊地址
      - 0.0.0.0：本机
      - 255.255.255.255：受限广播（不跨路由器）
      - 127.x.x.x：环回测试
      - 主机号全0：网络本身
      - 主机号全1：直接广播
    - 私有地址
      - A：10.0.0.0/8
      - B：172.16.0.0/12
      - C：192.168.0.0/16
  - 4 子网划分与CIDR
    - 子网掩码：1=网络位，0=主机位
    - CIDR：无分类编址，IP前缀/位数
    - 路由聚合：取共同前缀
    - 最长前缀匹配：多条匹配选前缀最长
    - 【易错】CIDR块大小 = 2^(32−前缀长度)
  - 5 NAT
    - 私网 ↔ 公网地址转换
    - NAPT：同时转换IP + 端口
    - 【易错】路由器对目的地址为私有地址的报文不转发
  - 6 核心协议 ★★★
    - ARP：IP → MAC
      - 请求广播、响应单播
      - 【易错】ARP是网络层协议
    - DHCP：动态分配IP
      - 应用层协议，基于UDP
      - 服务器67 / 客户68
      - 四步：Discover → Offer → Request → ACK
    - ICMP：差错报告 + 询问
      - 封装在IP中
      - ping = 询问报文；tracert = 差错报文
      - 不发差错报告的4种情况
        - 对ICMP差错报文本身
        - 第一个分片之后的后续分片
        - 组播地址
        - 特殊地址（127/0.0.0.0）
    - IGMP：组播管理，封装在IP中
  - 7 IPv6
    - 128位地址；固定首部40B
    - 表示：冒号十六进制；:: 只能出现一次
    - 【易错】取消首部校验和
    - 过渡：双协议栈、隧道技术
  - 8 路由协议对比 ★★★
    - RIP
      - 内部IGP；距离向量算法
      - 报文封装：UDP 520
      - 与相邻路由器交换整张路由表
      - 最大跳数15，16=不可达
      - 坏消息传得慢，收敛慢
    - OSPF
      - 内部IGP；链路状态 + Dijkstra
      - 报文封装：IP（协议号89）
      - 向全AS洪泛链路状态
      - 收敛快，无跳数限制
    - BGP
      - 外部EGP；路径向量
      - 报文封装：TCP 179
      - AS之间交换可达网络
  - 9 路由器
    - 路由选择部分（软件）+ 分组转发部分（硬件）
    - 功能：分组转发、路由选择、连接异构网络
    - 【易错】路由器隔离广播域，不转发广播帧
- 1 端口号
  - 0~1023熟知；1024~49151登记；49152~65535短暂
  - 必背
    - FTP数据20 / 控制21；SSH 22；Telnet 23
    - SMTP 25；DNS 53；DHCP 67/68
    - HTTP 80；POP3 110；BGP 179；HTTPS 443
  - Socket = IP + 端口
  - 【易错】端口号只有本地意义
- 2 UDP
  - 无连接、不可靠、面向报文、支持一对多/组播/广播
  - 首部8B：源端口 + 目的端口 + 长度 + 校验和
  - 【易错】校验含伪首部12B（参与计算不传输）
- 面向连接、可靠、面向字节流、全双工、点对点

- 六、树与二叉树 ★★★
  - 2 树的存储结构
    - 孩子表示法（链式）：每个结点一个孩子链表
    - 孩子兄弟表示法（二叉链表）：firstchild + nextbrother → 转为二叉树
  - 3 二叉树的概念
    - 二叉树是有序树
    - 特殊二叉树
      - 满二叉树
      - 完全二叉树
      - 二叉排序树：左子树 < 根 < 右子树
      - 平衡二叉树：任意结点左右子树深度差≤1
  - 4 二叉树的性质 ★★★
    - 一般树：结点数 = 所有结点度数之和 + 1
    - 二叉树
      - n0 = n2 + 1
      - 第k层至多 2^(k-1) 个结点
      - 高度为h至多 2^h − 1 个结点
    - 完全二叉树
      - 结点i所在层次 = floor[log2(i)] + 1
      - n个结点的完全二叉树高度 = ceil[log2(n+1)] 或 floor[log2(n)] + 1
      - 若有2k个结点：n0=k，n1=1，n2=k-1
      - 若有2k-1个结点：n0=k，n1=0，n2=k-1
    - m叉树
      - 第k层至多 m^(k-1) 个结点
      - 高度为h至多 (m^h−1)/(m−1) 个结点
      - 高度h与结点数n：(m^(h-1)−1)/(m−1) < n ≤ (m^h−1)/(m−1)
      - 即 h = ceil[log_m(n(m−1)+1)]
  - 5 二叉树的存储
    - 顺序存储：适合满/完全二叉树
      - 通常根下标从1开始：结点i的双亲 floor[i/2]、左孩子2i、右孩子2i+1
    - 二叉链表（链式存储）
      - n个结点 → 2n个链域，其中n-1个指针域被使用，n+1个空链域
  - 6 遍历
    - 先序/中序/后序：递归实现，非递归借助【栈】
    - 层序遍历：借助【队列】
    - 唯一确定一棵二叉树
      - 先序 + 中序 ✓
      - 后序 + 中序 ✓
      - 层序 + 中序 ✓
      - 【易错】先序 + 后序 ✗（只能确定结点祖先关系，无法区分左右子树）
  - 7 线索二叉树（链式存储）
    - 标志域：ltag=0 → lchild指示左孩子；ltag=1 → lchild指示前驱
      - rtag=0 → rchild指示右孩子；rtag=1 → rchild指示后继
    - 构造：按先序/中序/后序遍历实现对应的线索化
      - 可增设头结点：lchild指向根，rchild指向遍历最后一个结点
    - 遍历时找前驱 / 找后继
      - 中序：前驱✓ 后继✓
      - 先序：前驱× 后继✓（后继=有左孩子则先左孩子，否则右孩子）
      - 后序：前驱✓ 后继×（前驱=有右孩子则先右孩子，否则左孩子）
      - 【易错】先序不一定能直接找前驱、后序不一定能直接找后继，需借助三叉链表（知道双亲）才能一定找到
  - 8 树、森林与二叉树的转换
    - 树的遍历：先根遍历、后根遍历
    - 森林的遍历：先序遍历、中序遍历
    - 与二叉树遍历的对应
      - 树的先根遍历 = 对应二叉树的先序遍历
      - 树的后根遍历 = 对应二叉树的中序遍历
      - 森林的先序遍历 = 对应二叉树的先序遍历
      - 森林的中序遍历 = 对应二叉树的中序遍历
    - 转换的性质（森林/树 → 二叉树）
      - 左指针为空的个数 = 原来叶子结点个数
      - 右指针为空的个数 = 原来非叶结点数 + 1
      - 理解：①左指针空=没有孩子；②一组兄弟只对应一个空右指针，根结点无兄长故右指针也为空
  - 9 哈夫曼树 ★★
    - 定义：带权路径长度WPL最小的二叉树
    - 快捷计算WPL（非定义法）：所有非叶结点权值之和（仅适用于哈夫曼树）
    - 构造：每次选权值最小的两棵树合并，重复n-1次
    - 哈夫曼编码：前缀编码（任何一个编码都不是另一个编码的前缀）
    - m叉哈夫曼树：补充权值为0的虚结点，使 (n−1) % (m−1) == 0
  - 10 并查集
    - 存储结构：通常双亲表示法（顺序存储）
      - 每个集合一棵树；根结点下标 = 集合名
      - 根的双亲为负数（可取元素个数的相反数）
      - 所有集合的树构成森林
    - 操作
      - Initial(S)：初始化为单元素集合（双亲置-1）
      - Find(S, x)：找x所在集合的根（集合名）——时间与树的最大高度有关
      - Union(S, Root1, Root2)：Root2挂到Root1下；要求两集合不相交
      - Union本身O(1)，但需先Find
    - 优化
      - Find优化（压缩路径）：先找到根，再把查找路径上的结点直接挂到根下
      - Union优化（小树并入大树）：根结点双亲域存结点总数（负数），小树并大树
      - 两种优化后：树高 < log2(n)
    - 时间复杂度（Find / 合并n个独立元素）
      - 未优化：O(n) / O(n^2)
      - 仅Union优化：O(logn) / O(nlogn)
      - 全优化：O(α(n)) / O(n·α(n))
    - 应用
      - 判断无向图连通性：①每个顶点一个集合 ②按边两两合并 ③最终剩一个集合则连通
      - 判断图有环：合并过程中若某条边两端点已属同一集合 → 有环
      - 实现Kruskal：依次连接权值最小且属不同集合的两顶点，O(eloge)
- 1 基本概念
  - 有向图 & 无向图
  - 简单图（无自环、无重复边） & 多重图
  - 完全图
    - 无向完全图：C(2,n)条边
    - 有向完全图：2·C(2,n)条弧
  - 子图 & 生成子图（顶点集相同）
  - 连通性
    - 无向：连通图；连通分量 = 极大连通子图
    - 有向：强连通图；强连通分量
    - 【易错】连通无向图至少 n−1 条边；非连通无向图至多 C(2, n−1) 条边
    - 【易错】强连通有向图至少 n 条弧
  - 生成树 & 生成森林
    - 连通图的生成树：包含全部顶点的极小连通子图（n−1条边）
    - 砍去一条边则非连通，增加一条边则出现回路
    - 非连通图各连通分量的生成树 → 生成森林
  - 顶点的度、入度、出度
    - 无向图：度之和 = 边数×2
    - 有向图：入度和 = 出度和 = 边数
  - 带权图（网）；稠密图 & 稀疏图
  - 路径 & 回路；简单路径 & 简单回路（顶点不重复）
  - 有向树：一个入度为0，其余顶点入度均为1
- 2 存储结构
  - 邻接矩阵法（顺序存储）
    - 空间O(n^2)，适合稠密图
    - 【考点】A^n[i][j] = i到j长度为n的路径条数
  - 邻接表法（链式+顺序）
    - 空间：无向O(n+2e)；有向O(n+e)
    - 适合稀疏图
    - 【易错】有向图求出度易、求入度难（逆邻接表相反）
    - 【易错】邻接表表示不唯一；邻接矩阵唯一
  - 邻接多重表（无向图）
    - 空间O(n+e)；顶点顺序存储+边链式
    - 顶点结点（data, firstedge）；边结点（ivex, ilink, jvex, jlink）
  - 十字链表（有向图）
    - 空间O(n+e)
    - 顶点结点（data, firstin, firstout）；弧结点（tailvex, headvex, hlink, tlink）
- 3 图的遍历
  - BFS：借助辅助队列；DFS：递归（函数调用栈）
  - 时间复杂度
    - 邻接矩阵：BFS/DFS均O(n^2)
    - 邻接表：BFS/DFS均O(n+e)
  - 空间复杂度：O(n)（队列/栈最坏情况）
  - 应用
    - BFS求单源最短路径（仅无权图）
    - 判断无向图连通性、数连通分量个数（一次遍历不能访问全部 → 不连通）
    - 【易错】不能用一次遍历判断有向图强连通
    - 连通图遍历构造广度/深度优先生成树：邻接矩阵表示唯一，邻接表表示不唯一

- 五、传输层 ★★★
  - 3 TCP
    - 首部20B固定 + 最多40B选项
    - 【易错】SYN、FIN消耗1个序号；纯ACK不携带数据不消耗
    - 三次握手 ★
      - ① C → SYN=1, seq=x
      - ② S → SYN=1, ACK=1, seq=y, ack=x+1
      - ③ C → ACK=1, seq=x+1, ack=y+1
      - 为什么三次
        - 防止已失效的连接请求突然到达服务器
        - 同步双方初始序号
      - 【易错】第二次握手不能携带数据；第三次可以
    - 四次挥手 ★
      - ① C → FIN=1, seq=u
      - ② S → ACK=1, ack=u+1
      - ③ S → FIN=1, ACK=1, seq=w, ack=u+1
      - ④ C → ACK=1, ack=w+1 → TIME_WAIT等2MSL
      - 为什么四次：全双工，两个方向分别关闭
      - 2MSL原因
        - 保证最后一个ACK能被对方收到（可重传）
        - 让本连接所有旧报文在网络中消逝
      - 【易错】主动关闭方进入TIME_WAIT；被动方收到ACK直接CLOSED
    - 可靠传输
      - 机制：校验 + 序号 + 确认 + 重传
      - seq = 本报文段数据第一字节的序号
      - ack = 期望收到的下一字节序号
      - 超时重传：RTO略大于RTT（自适应）
      - 快重传：3个重复ACK → 立即重传
    - 流量控制
      - rwnd：接收方通告窗口
      - ★ 发送窗口 = min(rwnd, cwnd)
      - 零窗口：发送方停止；启动持续计时器 → 零窗口探测报文（防死锁）
    - 拥塞控制 ★★★
      - 慢开始：cwnd < ssthresh；每RTT翻倍（指数）
      - 拥塞避免：cwnd ≥ ssthresh；每RTT +1（线性）
      - 超时（严重拥塞）
        - ssthresh = max(cwnd/2, 2)
        - cwnd = 1；重新慢开始
      - 快重传：3个重复ACK → 立即重传
      - 快恢复
        - ssthresh = cwnd/2
        - cwnd = ssthresh；进入拥塞避免
      - 【易错】cwnd单位是MSS
      - 【易错】传输轮次 = 1个RTT
- 六、应用层
  - 1 模型
    - C/S：服务器常驻在线，可扩展性差
    - P2P：自扩展性强；每个节点既是客户又是服务器
  - 2 DNS ★
    - 域名结构：主机.次级.顶级.根
    - 服务器层级：根 → 顶级 → 权限 → 本地
    - 查询方式
      - 主机 → 本地服务器：递归查询
      - 本地服务器 → 其他：迭代查询
    - 端口53；主要UDP；区域传送用TCP
    - 【易错】DNS既用UDP又用TCP
  - 3 FTP
    - 控制连接21：全程保持
    - 数据连接20：按需建立，传完关闭
    - 【易错】带外传送：控制信息与数据分开
    - 主动PORT / 被动PASV
  - 4 电子邮件
    - 发信：SMTP 25（推协议）
    - 收信：POP3 110 / IMAP 143（拉协议）
    - 【易错】SMTP只能传7位ASCII；非ASCII用MIME
    - 【易错】用户→服务器、服务器→服务器都用SMTP；取件才用POP3/IMAP
  - 5 WWW与HTTP ★
    - URL：协议://主机:端口/路径；默认80
    - 特点：无状态、基于TCP、面向文本
    - HTTP/1.0：非持久连接
    - HTTP/1.1：持久连接（默认）
    - Cookie：弥补无状态
    - 报文：开始行 + 首部行 + 实体主体
    - 方法：GET / POST / HEAD / PUT / DELETE
      - HEAD：不返回实体主体，用于测试
  - 6 DHCP补充
    - UDP 67/68；应用层
    - Discover、Request为广播
    - 中继代理：跨网段分配
    - 租用期默认8天
- 七、易错陷阱30条（考前冲刺）
  - 1 曼彻斯特效率50%；10Mb/s→20MBaud
  - 2 Hub全不隔离；交换机只隔离冲突域；路由器全隔离
  - 3 最小帧长 = 2τ×C；10M以太网 = 64B = 512bit
  - 4 退避 k=min(次数,10)；16次放弃
  - 5 GBN发送 ≤ 2^n−1；SR收发都 ≤ 2^(n−1)
  - 6 海明码 2^r ≥ k+r+1
  - 7 IP首部长度单位4B，最大60B
  - 8 IP总长度单位1B，最大65535B
  - 9 片偏移单位8B
  - 10 TTL每过一个路由器减1
  - 11 协议字段：TCP=6，UDP=17，ICMP=1
  - 12 ARP：请求广播，响应单播
  - 13 DHCP是应用层，UDP 67/68
  - 14 ICMP封装在IP中
  - 15 RIP用UDP 520；OSPF用IP 89；BGP用TCP 179
  - 16 OSPF洪泛链路状态，不是整表
  - 17 TCP发送窗口 = min(rwnd, cwnd)
  - 18 拥塞超时：ssthresh=max(cwnd/2,2)，cwnd=1
  - 19 3重复ACK：ssthresh=cwnd/2，cwnd=ssthresh
  - 20 第三次握手可携带数据，消耗序号
  - 21 SYN、FIN各消耗1个序号
  - 22 TIME_WAIT等2MSL，主动关闭方
  - 23 DNS用UDP为主，区域传送用TCP
  - 24 FTP控制21全程开，数据20按需建
  - 25 SMTP发（25），POP3收（110）
  - 26 HTTPS = 443 = TCP + SSL
  - 27 ping是应用层，用ICMP，无端口
  - 28 IP分片只有最终目的主机重组
  - 29 CRC检错不能纠错；海明码纠1位
  - 30 IPv6无首部校验和；::只能出现一次
- 八、大题标准话术
  - 滑动窗口：发送方未收到确认前最多发送窗口大小数据量，接收方通告窗口实现流量控制
  - 三次握手：确认双方收发能力与初始序号，防止失效连接请求造成资源浪费
  - 拥塞控制：慢开始、拥塞避免、快重传、快恢复协同，动态调整cwnd；发送窗口=min(rwnd, cwnd)
  - 最小帧长：为保证发完一帧前能检测到冲突，发送时间 ≥ 争用期2τ，推出最小帧长 = 2τ × 数据率
  - ARP：广播请求（目的MAC全1），目的主机单播响应，源主机写入ARP缓存

- 1 概念与功能
  - 系统软件：管理计算机软硬件资源
  - 功能：处理机管理、存储器管理、文件管理、设备管理
  - 向上提供接口：命令接口（联机/脱机）、程序接口（系统调用）、GUI
- 2 特征 ★
  - 并发（最基本）、共享、虚拟、异步
  - 【易错】并发与并行：并行需要多处理机同时执行；并发是宏观同时微观交替

第一部分　数据结构（408）

- 七、图 ★★★
  - DFS实现逆拓扑排序：回溯（退栈）时输出顶点，再逆置得拓扑序列
  - 4 最小生成树（MST）★
    - 性质：权值之和最小
      - 通常不唯一（树形），但权和唯一
      - 【考点】各边权值互不相等时，MST唯一
    - Prim vs Dijkstra
      - 都是“依次选最近顶点”的贪心
      - Prim选距【生成树】最近的顶点 → 构造MST
      - Dijkstra选距【源点】最近的顶点 → 构造最短路径树（不一定是MST）
    - Prim vs Kruskal
      - 基本思想：Prim依次选距生成树最近的顶点 / Kruskal按权值递增选不同连通分量的边
      - 实现细节：Prim用isjoin数组+lowcost实时更新 / Kruskal用堆存边+并查集维护连通分量
      - 适用场合：Prim适合边稠密 / Kruskal适合边稀疏、顶点多
      - 时间复杂度：Prim O(n^2)（n轮，每轮O(n)选+更新） / Kruskal O(eloge)（堆取最小O(loge)，最坏选e次）
  - 5 最短路径 ★★
    - Dijkstra（单源）
      - 依次选距源点最近的顶点加入
      - dist[]记录源点到各点当前最短路长；path[]记录前驱（-1 = 直达/不可达）
      - 时间O(n^2)；求所有源点O(n^3)
    - Floyd（多源）
      - A(-1)=邻接矩阵；A(i) = 允许经过编号≤i的顶点中转后的最短路径矩阵
      - path矩阵记录路径的最后一个中转点（可递归展开整条路径）
      - A(i)对应的path(i)元素值 ≤ i
      - 时间O(n^3)，空间O(n^2)
    - 最短路径算法适用表
      - BFS：无权图√；单源；邻接矩阵O(n^2)/邻接表O(n+e)
      - Dijkstra：无权√带权√负权×负环×；单源O(n^2)
      - Floyd：无权√带权√负权√（无负环）×负环；多源O(n^3)
  - 6 有向无环图（DAG）
    - DAG描述表达式
      - 不存在回路的有向图；AOV网、AOE网都是DAG
      - 适合描述含公共子式的表达式（操作数只出现一次）
      - 步骤：①写操作数 ②按运算优先级自底向上分层写运算符 ③自底向上逐层合并
    - 拓扑排序（AOV网：顶点=活动，边=先后约束）
      - 拓扑排序：依次输出无前驱（入度0）的顶点
      - 逆拓扑排序：依次输出无后继（出度0）的顶点；拓扑序列的逆序一定是逆拓扑
      - 拓扑序列不唯一
      - 【考点】不能完成拓扑排序（无拓扑序列）⟺ 图中有环
      - 【考点】存在拓扑序列 ⟸ 邻接矩阵可重排为三角矩阵（反之亦成立）
      - 实现：①求各顶点入度 ②选入度0者输出并入队 ③出队并将其出边关联顶点入度减1，循环
      - DFS实现：回溯时输出得逆拓扑，栈倒置得拓扑
      - 复杂度：邻接矩阵O(n^2)；邻接表O(n+e)
    - 关键路径（AOE网：边=活动，权=活动耗时，顶点=事件）
      - 仅一个入度0的源点、一个出度0的汇点
      - 四个参量
        - ve(v)：事件v最早发生时间（源点到v最长路径）
        - vl(v)：事件v最迟发生时间（不拖延工期前提下）
        - e(a)：活动a最早开始时间 = 该弧起点的ve
        - l(a)：活动a最迟开始时间 = 弧终点的vl − 权值
        - d(a) = l(a) − e(a) = 时间余量
      - 关键活动：d=0的活动
      - 关键路径：源点到汇点权值和最大的路径 = 完成整个工程的最短时间
      - 【易错】加快关键活动可缩短工期，但缩短到一定程度后该活动可能变为非关键活动
      - 【易错】可能存在多条关键路径，只有加快【所有关键路径共有的】关键活动才能缩短工期
    - 图算法复杂度汇总
      - 邻接矩阵：Dijkstra O(n^2)、Floyd O(n^3)、Prim O(n^2)、DFS/BFS O(n^2)、拓扑/关键路径 O(n^2)
      - 邻接表：Kruskal O(eloge)、DFS/BFS O(n+e)、拓扑/关键路径 O(n+e)
- 八、查找 ★★★
  - 1 基本概念
    - 查找表：静态查找表（只查不改）/ 动态查找表（需插入删除）
    - 平均查找长度ASL（成功/失败）
  - 2 线性结构
    - 顺序查找
      - 优化：①元素按查找概率降序 → 优化ASL成功；②有序表 → 优化ASL失败
      - 适合顺序存储和链式存储
    - 折半查找
      - 要求元素有序；仅适合顺序存储（链表不行）
      - 判定树：n个元素查找成功最多比较ceil[log2(n+1)] 次
    - 分块查找（索引顺序查找）
      - 块内无序、块间有序；索引表记录各块最大关键字及起始地址
      - ASL = L(I)索引查找 + L(S)块内查找
      - 折半查找索引表时：low>high后，在low所指向的分块内查找
      - 【考点】n条记录分b块每块s条，s=√n 时ASL最小
  - 3 树形结构（查找树）
    - 二叉排序树BST
      - 定义：左子树 < 根 < 右子树（中序遍历递增）
      - 插入：新结点一定是叶子，是查找失败路径上最后访问结点的孩子
      - 删除
        - 叶子：直接删
        - 只有一棵子树：子树顶替
        - 两棵子树：用直接后继（或前驱）替代，再删后继
      - 查找效率：最好O(logn)（平衡），最坏O(n)（单边树）
    - 平衡二叉树AVL
      - 平衡因子 = 左子树高 − 右子树高 ∈ {−1, 0, 1}
      - 插入调整：找最小不平衡子树
        - LL → 右旋；RR → 左旋
        - LR → 先左旋后右旋；RL → 先右旋后左旋
      - 删除：按BST方式删除 → 从删除点向上找最小不平衡子树z → 看z的最高孩子y、y的最高孩子x → 按z/y/x排列选择LL/RR/LR/RL调整 → 若调整后子树高度改变，继续向上检查
      - 重要结论：非叶结点平衡因子均为1/−1时结点数最少
        - n(0)=0，n(1)=1，n(2)=2，n(h)=n(h-1)+n(h-2)+1
    - 红黑树 ★
      - 性质口诀：根叶黑、红相隔、黑高一致（叶为虚构空结点，bh=0）
      - 结论
        - 黑高相同（各路径黑结点数相同），红结点数 ≤ 黑结点数
        - 最长路径 ≤ bh×2（红黑相间）；最短路径 ≥ bh（全黑）
        - 最长路径 ≤ 最短路径×2（平衡性：左右子树高差不超过2倍）
        - h ≤ 2·log2(n+1)；查找效率O(logn)
      - 应用：C++ map/set、Java TreeSet/TreeMap
      - 插入：新结点红色
        - 若为根：染黑即可
        - 父为黑：无需调整
        - 父为红：看叔叔——口诀【黑叔跟着AVL，红叔换色再循环】
          - 黑叔：按AVL旋转调整（LL右旋 / RR左旋 / LR先左后右 / RL先右后左）
          - 红叔：父、叔变黑，爷变红；指针上移指向爷，爷视作新结点继续循环；若到根则染黑
      - 删除
        - 有孩子：找下位（中序前驱/后继）填补再删下位
        - 红色无孩：直接删
        - 黑色单孩：孩子必为红，顶替后染黑
        - 黑色无孩：产生双黑 → 恢复双黑看兄弟
          - 红兄：弑父并换色（红兄做父、原父做孩子、父兄颜色交换）→ 转为黑兄情形
          - 黑兄红孩：往上旋（旋转同AVL：LL/RR型直接结束，双黑恢复单黑；LR/RL型转为LL/RR型处理）
          - 黑兄黑孩：黑色上升——双黑的一层黑与黑兄的黑交给父亲；父红→染黑结束；父黑→产生新双黑，向上循环（仅初始为黑兄黑孩时可能循环到根）
    - 基本概念
      - B树的阶 = 所有结点孩子个数的最大值
      - B树的高 = 不包含外部结点（叶子）的层数
      - 内部结点 = 非叶结点（携带关键字）
      - 外部结点 = 叶结点（不含信息，查找失败）；均在同一层
      - 外部结点个数 = 关键字总个数 + 1
      - 终端结点 = 最底层的非叶结点
    - 每个结点至多m棵子树（至多m−1个关键字）

第三部分　操作系统（408）

- 一、操作系统概述
  - 3 发展
    - 单道批处理 → 多道批处理（提高资源利用率，无人机交互）
      - 【易错】多道程序设计的硬件基础：中断技术 + 通道技术
    - 分时系统：时间片轮转、交互性、响应及时（同时性、独立性）
    - 实时系统：可靠性、实时性（硬实时/软实时）
  - 4 运行机制
    - 内核程序 vs 应用程序；核心态 vs 用户态
    - 原语（关中断实现，不可中断）
    - 状态转换
      - 【易错】用户态→核心态：系统调用/中断/异常（陷入）
      - 【易错】切换由硬件完成；广义指令=系统调用
    - 中断分类（结合组成原理）
      - 内异常：缺页、系统调用、越界、除0
      - 外中断：时钟、DMA结束、键盘中断
    - 系统调用过程：传递参数→陷入指令→核心态执行→返回
      - 【易错】系统调用在核心态执行；系统调用发生在用户态，处理在核心态
- 二、进程与线程 ★★★
  - 1 进程概念
    - 程序的一次执行；资源分配的基本单位
    - 组成：PCB（进程存在的唯一标志）+ 程序段 + 数据段
      - PCB：进程描述信息、进程控制和管理信息、资源清单、处理机信息（现场）
    - 状态模型（五态）
      - 创建、就绪、运行、阻塞、终止
      - 运行→阻塞：主动（等I/O）；运行→就绪：被动（时间片到/剥夺）
      - 【易错】阻塞→运行不可能；就绪→阻塞不可能
  - 2 进程控制
    - 创建（fork）、终止（exit）、阻塞/唤醒（block/wakeup）
    - 均由原语实现
  - 3 进程同步 ★★★
    - 临界资源、临界区（访问临界资源的代码）
    - 同步：直接制约（合作进程次序）；互斥：间接制约（资源竞争）
    - 实现互斥
      - 软件法：单标志法（强制轮流）、Peterson（turn+flag，正确）
      - 硬件法：中断屏蔽、TestAndSet/Swap原子指令
    - 信号量机制（P、V）
      - P(wait)：S减1；S<0则阻塞入等待队列
      - V(signal)：S加1；S≤0则唤醒一个
      - 互斥：P(mutex)/V(mutex)夹住临界区
    - 经典问题 ★★★
      - 生产者-消费者：P(empty)/P(mutex)...V(mutex)/V(full)
        - 【易错】P(mutex)必须在P(资源信号量)之后，否则死锁
      - 读者-写者：count计数 + rw信号量 + mutex保护count
      - 哲学家进餐：奇数号先左后右（或限4人/一次全拿）
    - 管程：高级同步工具，同一时刻仅一个进程在管程内执行
  - 4 死锁 ★★★
    - 定义：互相等待对方资源，永远阻塞
    - 产生必要条件（4条缺一不可）
      - 互斥、持有并等待、不可剥夺、循环等待
    - 处理策略
      - 预防：破坏其中一条
        - 一次性申请全部（破持有等待）；可剥夺（破不可剥夺）；资源有序分配（破循环等待）
      - 避免：银行家算法
        - Available / Max / Allocation / Need = Max − Allocation
        - 安全序列：逐个找Need≤Available的进程，回收其Allocation
        - 【易错】不安全状态不一定死锁；死锁一定处于不安全状态
      - 检测与解除：资源分配图化简；剥夺、撤销、回退
  - 5 线程
    - 线程 = 调度的基本单位；进程 = 资源分配的基本单位
    - 用户级线程：内核看不见，切换快；一个阻塞全阻塞
    - 内核级线程：内核调度；切换需转入核心态
    - 【易错】线程共享进程的地址空间与资源；独享自己的栈、寄存器、PC
  - 6 处理机调度 ★★★
    - 三级调度：高级（作业→进程）、中级（挂起/激活，内存交换）、低级（进程切换）
    - 调度算法
      - FCFS先来先服务：对长作业有利、对短作业不利
      - SJF短作业优先：平均等待时间最短；可抢占（SRTF）
        - 【易错】SJF平均周转时间也最短（但饥饿）
      - 高响应比优先：响应比 = (等待+要求服务)/要求服务，兼顾长短
      - 时间片轮转：交互性好；时间片太大→FCFS，太小→切换开销大
      - 优先级调度：动态优先级防饥饿（等待越久优先级越高）
      - 多级反馈队列：综合最优（抢占、时间片逐级翻倍、动态优先级）
    - 性能计算
      - 周转时间 = 完成 − 到达；带权 = 周转/服务
      - 等待时间 = 周转 − 服务（不含I/O时）
  - 7 进程通信
    - 共享存储（直接/间接通信：消息缓冲、信箱）、管道（半双工、字节流）、消息传递、套接字
    - 【易错】管道通信：互斥+同步；读空管道阻塞
- 三、内存管理 ★★★
  - 1 程序装入与链接
    - 装入：绝对装入、可重定位（静态）、动态运行时装入（地址变换在运行时）
    - 链接：静态、动态（运行时、装入时）
  - 2 分区分配
    - 单一连续、固定分区（内部碎片）、动态分区（外部碎片）
    - 动态分区算法
      - 首次适应：空闲区按地址递增，从头找
      - 最佳适应：按容量递增 → 产生大量小碎片
      - 最坏适应：挑最大 → 防碎片但大作业难分配
      - 邻近适应（循环首次）：从上次位置找
      - 【易错】首次适应实际效果通常最好
    - 碎片：内部（分给作业没用完）、外部（太小无法分配）；紧凑技术消除外部碎片
  - 3 分页 ★★★
    - 页面与页框等大（2的幂，硬件决定）
    - 逻辑地址 = 页号 + 页内偏移
      - 【计算】页号 = 逻辑地址 / 页大小；偏移 = 逻辑地址 % 页大小
      - 【易错】页大小由硬件决定，页表长度与作业大小相关
    - 页表：页号→页框号
    - 地址变换：页号与页表长度比较（越界）→ 查页表 → 拼物理地址
    - 快表TLB
      - 【计算】有效访问时间EAT = α·(TLB+访存) + (1−α)·(TLB+页表+访存)
    - 多级页表：页表分页存储，减少页表本身占用
      - 【易错】多级页表增加一次访存（自内向外查）
    - 【易错】分页对程序员透明；内部碎片在最后一页
  - 4 分段与段页式
    - 分段：按逻辑单位划分，段长可变（二维地址）
      - 段表：段号→段基址+段长（判越界）
      - 共享与保护方便；外部碎片
    - 分段 vs 分页：页是物理单位对用户透明；段是逻辑单位对用户可见
    - 段页式：先查段表再查页表（3次访存：段表、页表、数据）
    - 【易错】段页式地址结构：段号+段内页号+页内偏移
  - 5 虚拟内存 ★★★
    - 基础：局部性原理；特征：多次性、对换性、虚拟性
    - 请求分页
      - 页表增加：状态位（是否在内存）、访问位、修改位（脏页换出需写回）、外存地址
      - 缺页中断：指令执行期间产生、属于内异常
        - 【易错】缺页中断处理期间进程阻塞；一条指令可能引发多次缺页
      - 调入：请求调页；预调页
    - 页面置换算法 ★★★
      - OPT最佳置换（向后看，理论基准，无法实现）
      - FIFO先进先出：会出现Belady异常
        - 【易错】Belady异常仅FIFO会出现（物理块增多缺页反增）
      - LRU最近最久未使用：硬件支持代价大（计数器/栈）
      - Clock（NRU）简单时钟：访问位1/0，二次机会
      - 改进Clock：考虑修改位，优先淘汰(访问位,修改位)=(0,0)的页
    - 抖动（颠簸）：缺页率过高，CPU大部分时间在换页
      - 原因：分配的物理块不足 / 工作集增大
      - 工作集：一段时间内实际访问页的集合；驻留集≥工作集防抖动
    - 【易错】虚拟内存容量 ≤ min(内存+外存, 机器寻址范围)
- 四、文件管理 ★★★
  - 1 文件逻辑结构
    - 顺序文件（串结构/顺序结构）、索引文件（定长/变长+索引表）、索引顺序文件
    - 【考点】索引顺序文件：√n分组长查询最快
  - 2 目录
    - FCB文件控制块（文件名+属性）→ 目录项
    - 目录结构：单级、两级、树形、无环图（共享）
    - 索引结点inode：目录项仅存文件名+inode号（目录瘦身，检索快）
    - 路径名、当前目录
  - 3 文件物理结构 ★★★
    - 连续分配
      - 支持随机访问、顺序访问快；难扩展、外部碎片
    - 链接分配
      - 隐式链接：每块存下一块指针；只能顺序访问；指针开销（改进：簇）
      - 显式链接：FAT表（内存中，查表即得全部块号）；随机访问
    - 索引分配
      - 单级索引：索引表太长放不下
      - 多级索引、链接方案
      - 混合（增量）索引：UNIX inode
        - 直接地址（小文件快）+ 一次间接 + 二次间接 + 三次间接
        - 【计算】文件最大长度 = 各级可寻址块数×块大小求和
    - 【易错】隐式链接FAT属于显式链接；FAT开机后常驻内存
  - 4 存储空间管理
    - 空闲表、空闲链表（盘块/连续区）、位示图
      - 【计算】位示图字号位号 ↔ 盘块号
    - 成组链接（UNIX）：空闲盘栈+成组（少量额外空间管理大量空闲块）
  - 5 文件共享与保护
    - 硬链接：同一inode，引用计数；断链减计数
    - 软链接（符号链接）：存路径，快捷方式，多一次寻径
    - 保护：ACL访问控制列表、口令、密码

- B树（多路平衡查找树）★
  - m阶B树性质
    - 根结点非终端时至少2棵子树
    - 非根非叶结点至少⌈m/2⌉棵子树（至少⌈m/2⌉−1个关键字）
  - 高度h与关键字数n
    - 关键字尽量多（高度尽量小）：n ≤ m^h − 1，即 h ≥ log_m(n+1)
    - 关键字尽量少（高度尽量大）：n+1 ≥ 2·⌈m/2⌉^(h-1)
  - B树查找：①在树中找结点（磁盘→读入内存）②结点内找关键字（内存，顺序或折半）
  - 【易错】B树不能有效支持顺序查找
  - 插入（分裂）
    - 插入位置 = 查找失败的终端结点
    - 插入后关键字数 > m−1 → 从中间位置分裂，中间关键字上移（可能连锁分裂）
  - 删除（合并）
    - 不在终端结点：用其前驱/后继替代，转化为删终端结点中的关键字
    - 在终端结点
      - 删后仍满足关键字下限 → 直接删除
      - 兄弟够借 → 左/右兄弟上移一个关键字到双亲，双亲对应关键字下移（旋转）
      - 兄弟不够借 → 与兄弟及双亲中对应关键字合并；合并使双亲关键字减少，可能继续向上调整；若双亲是根且减空 → 删根，子结点成新根，树高减1
- B+树 ★
  - 应用于数据库，是B树的一种变形
  - n个关键字 → n棵子树（每个关键字对应一棵子树）
  - 结点划分
    - 内部结点（分支结点）：只起索引作用，存放子结点关键字的最大值
    - 叶结点：包含全部关键字及指向记录的指针
  - 关键字个数：根2 ≤ n ≤ m；非根⌈m/2⌉ ≤ n ≤ m
  - 查找：无论成功失败最终都走到叶结点
  - 【考点】B+树支持顺序查找和多路（随机）查找；B树不支持顺序查找
  - 插入删除与B树类似
- 4 散列结构 ★★★
  - 概念
    - 散列表、散列函数
    - 同义词：映射到同一地址的不同关键字
    - 冲突；聚集/堆积：非同义词争夺同一后继哈希地址
  - 散列函数构造
    - 直接定址法：H(key)=a·key+b；不冲突但浪费空间；适合关键字分布基本连续
    - 除留余数法：H(key)=key % p；p取不大于表长的素数
    - 数字分析法：取分布均匀的若干位；适合已知关键字集合
    - 平方取中法：取平方结果的中间几位；适合关键字各位取值不均匀
  - 冲突处理
    - 开放定址法：Hi = (H(key)+di) % m；空闲地址对同义词和非同义词都开放
      - 线性探测法：di=0,1,2,...；易产生堆积
      - 平方探测法（二次探测）：di=0²,1²,−1²,...；表长必须是4k+3形式的素数；减轻堆积；不能探测全部单元（至少一半）
      - 再散列法（双散列）：Hi=(H(key)+i·Hash2(key)) %m；Hash2计算探测增量；最多m−1次即可遍历所有位置回到H0
      - 伪随机序列法：di取伪随机序列
      - 【易错】开放定址法不能随便物理删除（会截断同地址元素的探测链）→ 做删除标记逻辑删除，定期维护物理删除
    - 拉链法：所有同义词链在一个线性链表；完全避免聚集
  - 性能分析
    - 以ASL衡量；影响因素：①散列函数 ②冲突处理方法 ③装填因子
    - 装填因子 α = 表中记录数n / 散列表长度m；α越大越容易冲突
    - ASL失败：探测到空单元计入次数；拉链法访问空指针不计入
- 九、排序 ★★★
  - 1 基本概念
    - 内部排序 / 外部排序
    - 算法稳定性（关键字相同的元素排序后相对次序不变）
  - 2 内部排序算法总对比 ★★★
    - 直接插入：最好O(n)、平均/最坏O(n²)；空间O(1)；稳定
      - 适合基本有序或n小
    - 折半插入：O(n²)（比较O(nlog2n)，移动仍O(n²)）；空间O(1)；稳定
    - 希尔：最坏O(n²)，约O(n^1.3)；空间O(1)；不稳定；仅顺序存储
    - 冒泡：最好O(n)、平均/最坏O(n²)；空间O(1)；稳定
    - 快速：平均/最好O(nlogn)、最坏O(n²)（基本有序时）；空间O(logn)、最坏O(n)；不稳定
      - 内部排序中平均性能最优
    - 简单选择：O(n²)；空间O(1)；不稳定；移动次数最少（≤n−1次交换）
    - 堆：建堆O(n)+排序O(nlogn)；空间O(1)；不稳定
    - 归并：O(nlogn)；空间O(n)；稳定
    - 基数：O(d(n+r))；空间O(r)；稳定
    - 选择建议
      - n小：直接插入或简单选择（每条记录信息量大时选简单选择，比较次数固定但移动少）
      - 基本有序：直接插入或冒泡
      - n大：快排（平均最快）、堆排（空间少、无最坏陷阱）、归并（稳定）
      - 【考点】n个关键字随机分布时，任何基于比较的排序下界为O(nlogn)
      - 记录移动开销大 → 链表存储（归并/基数/冒泡等）
  - 3 插入排序
    - 直接插入：将元素插入前面已排序序列
    - 折半插入：查找用折半，移动不变
    - 希尔排序：增量分组直接插入，增量递减至1
  - 4 交换排序
    - 冒泡：每趟沉底一个最大值；最好初始正序一趟完成O(n)
    - 快速排序
      - 一趟划分Partition：枢轴归位，左边≤枢轴、右边≥枢轴
      - 【易错】一趟排序 ≠ 一次划分；一趟后未必"整体大于/小于"相邻
      - 【易错】Partition与枢轴比较条件为≤或≥（写代码细节）
  - 5 选择排序
    - 简单选择：每趟选最小与当前位置交换
    - 堆排序
      - 建初始堆：从最后一个非叶结点（第⌊n/2⌋个）开始，从后往前逐个下坠调整；O(n)
      - 基本操作（对堆）
        - 删除O(logn)：最后一个元素顶替堆顶，向下调整（下坠）
        - 插入O(logn)：插入尾部，自下往上（上升）调整
      - 排序流程：建初始堆 → 输出堆顶（与末尾交换）→ 调整为新堆 → 重复n轮；O(nlogn)
      - 【易错】关键字比较次数：下坠时有2个孩子比较2次、1个孩子比较1次；上升时比较1次
  - 6 归并排序
    - N个元素k路归并的趟数：k^m = N → m = ceil(log(k, N))
    - 每趟时间复杂度O(n)
  - 7 基数排序
    - 基于多关键字（位）：位数d、基数r（十进制r=10）
    - MSD最高位优先：先按高位分组，组内再按次高位分组，d趟分配后依次收集
    - LSD最低位优先（更好）：d趟分配+收集；利用稳定性，高权位相等时不影响已按低位确定的相对次序，故每次分配后可直接收集
  - 8 外部排序
    - 多路归并排序：k路归并，r个初始段需 ceil(log(k, r)) 趟
    - 多路平衡归并与败者树
      - 败者树使k路归并选最小只需O(log2k)次比较
    - 置换-选择排序（生成初始归并段）：平均段长约为内存容量的2倍
    - 最佳归并树：类似哈夫曼树；若(段数−1)%(k−1)≠0需补虚段0
- 十、数据结构易错陷阱30条（考前冲刺）
  - 1 线性表是逻辑结构，顺序表/链表是存储结构；栈/队列同理
  - 2 循环队列牺牲单元判满：(Q.rear+1)%MaxSize==Q.front；队空front==rear
  - 3 n个元素入栈出栈序列数 = 卡特兰数 = C(2n,n)/(n+1)
  - 4 共享栈判满：top1−top0==1（两栈顶相邻）
  - 5 队列判满3法：牺牲单元 / Q.size / tag法
  - 6 零个元素的空串长度0；空格串≠空串
  - 7 KMP的next是PM表整体右移，next[0]=−1；下标从1时整体+1用0代表−1
  - 8 中序线索找前驱后继都行；先序找不到前驱、后序找不到后继（需三叉链）
  - 9 任意二叉树：n0 = n2 + 1
  - 10 高h完全二叉树结点数范围：2^(h-1) ≤ n ≤ 2^h −1
  - 11 二叉链表n结点 → n+1个空指针域
  - 12 先序+后序不能唯一确定二叉树
  - 13 树的先根 = 二叉树先序；树的后根 = 二叉树中序
  - 14 树转二叉树：左空指针数=叶子数；右空指针数=非叶数+1
  - 15 哈夫曼树WPL = 所有非叶结点权值之和（快捷法）
  - 16 m叉哈夫曼：补(虚段)满足(n−1)%(m−1)=0
  - 17 无向图连通至少n−1条边；非连通至多C(2,n−1)条边
  - 18 邻接矩阵的k次幂A^k[i][j] = i到j长度为k的路径数
  - 19 无向图邻接表空间O(n+2e)（每条边存两次）
  - 20 BFS只解决无权最短路径；不能一次遍历判定有向图强连通
  - 21 无拓扑序列 ⟺ 有环；拓扑序列唯一不能推出图无环（反之也不行）
  - 22 关键路径是最长路径；缩短工期必须缩短所有关键路径共有的关键活动
  - 23 Dijkstra不能处理负权；Floyd可处理负权边但不能有负环

- 五、输入/输出管理 ★★
  - 1 I/O层次结构
    - 用户层I/O（库函数）→ 设备独立性软件（系统调用、命名、保护、缓冲）→ 设备驱动程序 → 中断处理程序 → 硬件
    - 【易错】设备驱动程序是唯一知道设备细节的层次
  - 2 I/O核心子系统
    - I/O调度、设备保护、缓冲
    - 缓冲区 ★★
      - 单缓冲：每块处理时间 = max(C, T) + M
      - 双缓冲：max(T, C+M)；交换时T与M并行
      - 循环缓冲、缓冲池（多个进程共享）
      - 【易错】缓冲区属设备独立性软件层；缓冲区在内存
    - SPOOLing（假脱机）★
      - 用输入井/输出井模拟独占设备的共享
      - 独占设备 → 变共享（如打印机排队）
      - 组成：井、输入/输出缓冲区、输入/输出进程
  - 3 设备分配
    - 独占/共享/虚拟设备；DCT设备控制表→COCT控制器→CHCT通道
    - 设备分配安全性（死锁避免）
  - 4 磁盘管理 ★★★
    - 磁盘地址：柱面（磁道）+ 磁头（盘面）+ 扇区
    - 访问时间 = 寻道时间 + 旋转延迟（平均半圈）+ 传输时间
      - 【计算】旋转延迟 = (60/转速)/2
    - 调度算法 ★★★
      - FCFS：公平，平均寻道长
      - SSTF最短寻道优先：局部最优，可能饥饿
      - SCAN电梯（往返扫描）：到边界折返
      - LOOK：SCAN改进，到最远请求即折返（不必到边界）
      - CSCAN单向循环：磁道均匀分布服务更公平
      - CLOOK：CSCAN+LOOK
      - 【易错】SCAN过终点，LOOK不过终点；题目只给请求序列默认LOOK也常按SCAN算端点
    - 磁盘管理：格式化、坏块、分区
- 六、操作系统易错陷阱30条（考前冲刺）
  - 1 并发是并行的子集：单CPU只能并发不能并行
  - 2 系统调用执行于核心态；陷入指令（trap）是用户态唯一主动进入核心态的方式
  - 3 用户态到核心态由硬件完成切换，OS无法直接执行特权指令前切换
  - 4 PCB是进程存在的唯一标志；进程与程序的本质区别是动态性
  - 5 运行→阻塞是主动行为（等I/O）；运行→就绪是被动剥夺
  - 6 不存在阻塞→运行、就绪→阻塞的直接转换
  - 7 子进程fork返回0，父进程返回子进程PID（Linux）
  - 8 P操作顺序：先资源信号量后互斥信号量，否则死锁
  - 9 V操作不会阻塞（只可能唤醒别人）
  - 10 死锁必要条件4条缺一不可；不安全状态≠死锁
  - 11 银行家算法：Need = Max − Allocation
  - 12 资源有序分配破坏"循环等待"
  - 13 线程是调度单位，进程是资源分配单位；线程切换不一定引起进程切换
  - 14 线程独享栈和寄存器，共享地址空间
  - 15 多级反馈队列可抢占：新进程进高优先级队列可抢占低级队列执行者
  - 16 SJF平均等待时间最短但饥饿长作业
  - 17 带权周转时间≥1
  - 18 动态分区产生外部碎片；固定分区产生内部碎片
  - 19 最佳适应产生最多小外部碎片；实际效果首次适应最好
  - 20 分页地址：页号=逻辑地址/页大小；页内偏移不进页表
  - 21 分页对用户透明；分段对用户可见（二维）
  - 22 访存次数：基本分页2次（页表+数据）；一级TLB命中则1次；二级页表无TLB为3次
  - 23 段页式无TLB要3次访存（段表→页表→数据）
  - 24 缺页中断在指令执行期间产生（不是指令末尾），一条指令可多次缺页
  - 25 Belady异常只有FIFO；LRU/OPT无此异常
  - 26 改进Clock优先淘汰（访问位,修改位)=(0,0)的页
  - 27 抖动主因：物理块分配不足；解决：调工作集
  - 28 UNIX混合索引：一次间接块容量=块大小/地址项大小
  - 29 位示图：字号i位j对应盘块号 = i×字长 + j（按0起）
  - 30 电梯算法：SCAN到达磁道终点折返；LOOK到最远请求折返
- 七、大题标准话术
  - PV设计：先确定互斥资源与协作次序，画前趋图，再为每个约束设信号量；互斥P(mutex)紧贴临界区，资源P在前；每个P对应一个V
  - 银行家算法：列出Available、Max、Allocation、Need矩阵，尝试安全序列；能回收完成进程的资源进入下一轮判定，全进程可完成则安全
  - 请求分页计算：逐条指令访问→页号→是否在内存；未命中则按置换算法选牺牲页，若其修改位=1还需写回（计一次缺页+可能的I/O）
  - 磁盘调度：当前磁道按给定算法依次选下一访问磁道，累计移动道数；SCAN先判断当前移动方向
  - 地址翻译：逻辑地址二进制划分（页号P位+偏移W位），查页表得页框号，拼接物理地址；注意TLB命中路径

- 24 Prim适合稠密$O(n^2)$；Kruskal适合稀疏O(eloge)
- 25 平衡树最少结点递推：n(h)=n(h-1)+n(h-2)+1
- 26 红黑树最长路径≤最短路径×2；h≤2log2(n+1)
- 27 m阶B树根非终端至少2棵子树；非根非叶至少⌈m/2⌉棵
- 28 开放定址法删除只能做标记（逻辑删除）
- 29 各排序算法稳定性：不稳的有希尔/快排/简单选择/堆
- 30 快排最坏情况：初始序列基本有序/基本逆序，退化为$O(n^2)$
- 十一、大题标准话术
  - Dijkstra：按源点到各顶点最短路径长度递增次序逐个加入，每次用新加入顶点dist更新其邻接点，n−1轮后全部确定
  - Floyd：从邻接矩阵出发，逐个将顶点作为中转点松弛：若A[i][k]+A[k][j] < A[i][j]则更新并记录中转点
  - 关键路径：先正推求ve，再逆推求vl；e(a)=ve(弧尾)，l(a)=vl(弧头)−权；d=0的活动即关键活动，由关键活动构成源点到汇点的最长路径
  - 拓扑排序：反复选择入度为0的顶点输出并删除其出边；若输出顶点数少于总顶点数，则图存在回路
  - 缺页/散列ASL：逐个计算每个关键字查找成功所需比较次数取平均；失败按每个地址开始探测到空单元为止
  - 红黑树插入：口诀"黑叔跟着AVL，红叔换色再循环"——黑叔按LL/RR/LR/RL旋转，红叔父叔变黑爷变红继续向上

- 一、计算机系统概述
  - 1 层次结构
    - 冯·诺依曼机
      - 核心：存储程序（指令+数据同等地位存于存储器，按地址访问）
      - 五大部件：运算器、控制器、存储器、输入设备、输出设备
      - 指令 = 操作码 + 地址码，顺序执行
      - 【易错】早期以运算器为中心；现代以存储器为中心
    - 硬件组成
      - 运算器：ALU、ACC、PSW等
      - 控制器：PC、IR、MAR、MDR、控制单元CU
      - 【易错】MAR/MDR属于存储器还是CPU：现代属于CPU（对用户透明）
    - 层次结构（由下至上）
      - 硬件 → 微程序机器（微指令）→ 机器语言层（指令集）→ 操作系统层 → 汇编语言层 → 高级语言层
      - 软硬件分界面：指令集体系结构ISA
      - 翻译/解释：汇编语句→机器语句一对一翻译；高级语言一对多
  - 2 性能指标 ★★
    - 机器字长：CPU一次能处理的位数
    - CPU性能
      - 主频 = 时钟周期倒数
      - CPI：执行一条指令所需时钟周期数
      - CPU执行时间 = 指令条数 × CPI × 时钟周期
      - IPS = 主频 / CPI；MIPS、FLOPS
    - 数据通路带宽、吞吐量、响应时间
    - 【易错】主频高的CPU性能不一定强（CPI、指令数都影响）
    - 【易错】基准程序测试不能完全反映一台机器的性能
- 二、数据的表示和运算 ★★★
  - 1 进位计数制
    - 二进制优点：物理实现容易、运算规则简单、与逻辑值对应
    - 转换
      - 任意进制→十进制：按权展开
      - 十进制→R进制：整数除R取余（余数倒序），小数乘R取整
      - 【易错】十进制小数可能无法精确转为二进制（如0.3）
  - 2 定点数表示
    - 原码：符号位+绝对值；0有两种表示
    - 反码：负数取反；0有两种表示
    - 补码：反码+1；0唯一表示
      - 【快捷】由[X]补求[−X]补：连同符号位一起取反加1
      - 表示范围：8位补码 −128 ~ +127（−128无原码）
    - 移码 = 真值 + 偏置（2^(n-1)）；用于阶码比较大小
      - 【易错】移码大→真值大；移码0表示最小值
    - 【易错】位数扩展
      - 正数补码补0；负数补码符号扩展补1
  - 3 定点运算
    - 移位
      - 算术移位：符号位不变（原码数值移位；补码负数左移补0右移补1）
      - 逻辑移位：无符号数，统一补0
    - 溢出判断（补码加减）
      - 一位符号位：最高数值位进位 ⊕ 符号位进位 = 1则溢出
      - 双符号位（变形补码）：两符号位不同则溢出
      - 【易错】正溢出/负溢出：双符号位01正溢、10负溢
    - 乘除：原码一位乘（符号异或）、Booth补码一位乘
    - 【易错】补码加减运算连符号位一起参加运算
  - 4 浮点数与IEEE 754 ★★★
    - 格式
      - 单精度：1符号 + 8阶码 + 23尾数
      - 双精度：1符号 + 11阶码 + 52尾数
    - 规格化
      - 阶码真值 = 阶码字段 − 偏置值（127 / 1023）
      - 尾数隐含最高位1：1.M（省略存储，多出1位精度）
      - 范围：单精度阶码真值 −126 ~ +127（规格化数）
    - 特殊值
      - 阶码全1、尾数0 → ±∞
      - 阶码全1、尾数≠0 → NaN
      - 阶码全0 → 非规格化数（隐含位为0，阶码真值取 −126）
    - 浮点加减5步（必背顺序）
      - ①对阶：小阶向大阶，小阶尾数右移
      - ②尾数加减
      - ③规格化：左规（尾数溢出/非规格化时右规）
      - ④舍入：0舍1入 / 恒置1 / 向偶舍入
      - ⑤溢出判断：阶码溢出才真溢出
    - 【易错】对阶永远小阶向大阶（小数右移损失精度）
    - 【易错】尾数溢出不等于浮点溢出（可右规）
    - 【易错】浮点数范围比补码大得多，但精度受限
  - 5 运算电路
    - 一位全加器：本位和 = A⊕B⊕Cin；进位 = AB + (A⊕B)Cin
    - 串行加法器慢；先行进位加法器（并行）快
    - ALU（74181）：组合逻辑，核心是加法器
  - 6 校验码 ★★
    - 码距d：检错能力d−1，纠错能力⌊(d−1)/2⌋
    - 奇偶校验：码距2，检1位错不能纠错
    - 海明码：纠1位错
      - r位校验：2^r ≥ k + r + 1
      - 校验位放在2^i位置
    - CRC循环冗余校验：模2除法求余数（余数即FCS附于帧尾）
      - 【易错】CRC能检错不能纠错（数据链路层面）
- 三、存储器层次结构 ★★★
  - 1 层次结构
    - Cache—主存：解决速度（硬件实现，对程序员透明）
    - 主存—辅存：解决容量（OS+硬件=虚拟存储器）
    - 【易错】存储层次越高越快越贵、容量越小
  - 2 半导体存储器
    - SRAM：触发器存储；快、贵、不需刷新；用于Cache
    - DRAM：电容存储；慢、便宜、需定期刷新（1~2ms）
      - 刷新方式：集中刷新（死时间）、分散刷新（无死时间但存取周期变长）、异步刷新（折中）
      - 【考点】刷新以行为单位
    - ROM：PROM、EPROM、EEPROM、Flash（U盘/SSD）
    - 【易错】Flash写入前必须先擦除
  - 3 主存组织
    - 芯片扩展：位扩展（加大字长）、字扩展（加大字数）
    - DRAM地址复用：分行/列两次传送（RAS行选通、CAS列选通），引脚减半
    - 多体交叉存储器
      - 高位交叉：顺序存储，一个体内连续，无并行加速
      - 低位交叉：流水存取，连续地址分布在不同体
      - 【考点】m体低位交叉流水：启动间隔 T/m；连续存m个字时间 T + (m−1)·T/m
  - 4 Cache ★★★
    - 基本原理：程序访问的局部性原理
      - 时间局部性：刚访问过的很快再访问（循环）
      - 空间局部性：邻近的将被访问（数组顺序）
    - 命中率与平均访问时间
      - h = Nc/(Nc+Nm)
      - 平均访问时间 = h×Tc + (1−h)×(Tc+Tm)
    - 地址映射
      - 直接映射：行号 = 主存块号 mod Cache行数；冲突率高、硬件简单
      - 全相联：任意行；无冲突灵活，比较器多成本高
      - 组相联：组间直接映射、组内全相联
      - 【易错】直接映射无需替换算法；全相联/组相联需要
    - 替换算法：LRU（栈实现）、LFU、FIFO、随机
    - 写策略
      - 写命中：全写法（写直达，配写缓冲）/ 回写法（脏位，减少访存）
      - 写不命中：写分配法（载入块再写，配回写法）/ 非写分配法（直接写主存，配全写法）
    - 【易错】Cache对程序员完全透明；Cache内容是主存副本
    - 【计算】CPU平均访存时间 = 访Cache命中次数×Tc + 未命中次数×(Tc+Tm)
  - 5 虚拟存储器（结合操作系统）
    - 页式虚拟存储：逻辑地址→页表→物理地址
    - TLB快表：存常用页表项（页号+页框号）
    - 三级查找：TLB → Page（内存）→ 主存数据
    - 【计算】访存次数：TLB命中1次；TLB未命中查页表+取数2次；缺页则更多

- 6 磁盘
  - 磁盘地址：柱面号 + 磁头号 + 扇区号
  - 存取时间 = 寻道时间 + 旋转延迟（平均半圈） + 传输时间
  - 【易错】磁盘转速×2 = 旋转一圈时间倒数

第二部分 计算机组成原理（408）

- 四、指令系统
  - 1 指令格式
    - 指令 = 操作码 + 地址码
    - 地址码结构：零/一/二/三/四地址指令
    - 扩展操作码：短码留给高频指令
      - 【计算】操作码按等长扩展：15/15/15/16型分配
    - 【易错】地址码个数少 → 指令字长短，但访存次数多（累加器隐含）
  - 2 寻址方式 ★★★
    - 指令寻址
      - 顺序寻址：PC + "1"（一条指令字长）
      - 跳跃寻址：转移指令修改PC
    - 数据寻址
      - 立即寻址：操作数在指令中（最快）
      - 直接寻址：地址码=有效地址EA（访存1次）
      - 间接寻址：地址码指向EA（访存≥2次，可扩大范围）
      - 寄存器寻址：操作数在寄存器（不访存）
      - 寄存器间接寻址：寄存器存EA（访存1次）
      - 相对寻址：EA = PC + 偏移（转移指令，位置无关）
      - 基址寻址：EA = 基址寄存器 + 偏移（面向OS，扩大寻址、程序重定位）
      - 变址寻址：EA = 变址寄存器 + 偏移（面向用户，数组遍历）
      - 堆栈寻址：SP隐含
    - 【易错】相对寻址取指令后PC已+1，EA以新PC为基准
    - 【易错】基址内容不变变的是偏移（操作系统）；变址内容变偏移固定（处理数组）
  - 3 CISC vs RISC ★
    - CISC：指令多、长度可变、微程序控制、访存不受限
    - RISC：指令少而规整、load-store结构寄存器操作、硬布线+流水、指令定长
    - 【易错】RISC 不一定更快，但更适合流水线
- 五、中央处理器 ★★★
  - 1 CPU结构
    - 运算器 + 控制器
    - 用户可见寄存器：通用寄存器、ACC、PSW（条件标志）
    - 用户不可见：PC（指令指针）、IR、MAR、MDR
    - 【易错】PC存下条指令地址，取指后自动+1；转移指令才显式修改PC
  - 2 指令执行过程
    - 指令周期 = 取指周期 → [间址周期] → 执行周期 → [中断周期]
    - 数据通路：寄存器间传送的路径（专有数据通路/分时复用总线）
    - 【易错】一条指令的执行可能包含多次访存（间址、操作数）
  - 3 控制器
    - 硬布线：组合逻辑直接生成控制信号；速度快，RISC用
    - 微程序：控制存储器（ROM）存微指令；规整易扩展，CISC用
      - 微指令格式：水平型（并行强、字长长）/ 垂直型（类似指令）
      - 微地址形成：增量方式 / 断定方式（测试字段）
    - 【易错】微程序存于控制器内的控制存储器（ROM），对程序员透明
  - 4 指令流水线 ★★★
    - 经典5段：取指IF → 译码ID → 执行EX → 访存MEM → 写回WB
    - 流水线性能
      - 完成n条指令时间 = (k + n − 1) × 时钟周期
      - 吞吐率、加速比（极限k倍）
    - 三大冒险/冲突
      - 结构冒险：硬件资源冲突（取指与取数同访存 → 指令Cache/数据Cache分离）
      - 数据冒险：后续指令用到前面指令结果（RAW）
        - 解决：暂停stall / 转发forwarding（旁路）/ 编译调度
        - 【易错】load-use冒险无法完全转发，需1个stall
      - 控制冒险：转移指令改变PC
        - 简单处理：冻结/排空流水线（2~3周期损失）
        - 预测：静态（预测不转移/循环向后）、动态（历史1位/2位状态机）
    - 高级技术
      - 超标量：多条流水线并行、多发射（空间并行）
      - 超流水：更细分段提高频率（时间并行）
      - 超长指令字VLIW：编译器静态调度
    - 【易错】5段流水线每个功能段都要1个时钟周期
  - 5 异常与中断
    - 内异常（CPU内部、同步）
      - 故障fault：缺页、越界（可修复重新执行）
      - 陷入trap：系统调用、断点（有意）
      - 终止abort：硬件错误
    - 外中断（外设、异步）：可屏蔽INTR、不可屏蔽NMI
    - 【易错】系统调用 = 陷入（内异常）；DMA结束、时钟 = 外中断
    - 【易错】缺页异常由硬件检测、OS处理
- 六、总线
  - 1 概念
    - 分类
      - 按位置：片内总线、系统总线、通信总线
      - 按功能：数据总线（双向）、地址总线（单向）、控制总线
    - 总线事务：请求→仲裁→寻址→传输→释放
    - 【易错】单总线结构中CPU、主存、外设都挂同一总线
  - 2 性能指标
    - 总线带宽 = 总线工作频率 × 总线宽度(bit) / 8
    - 【计算】DDR等协议倍率注意×2
  - 3 仲裁
    - 集中式：链式查询（BG串行，优先级固定离控制器近的高）、计数器定时（循环优先）、独立请求（最快最贵）
    - 分布式：自举分配等
  - 4 定时
    - 同步定时：统一时钟，速度快但必须迁就最慢设备
    - 异步定时：握手信号（不互锁/半互锁/全互锁）
    - 半同步：同步+等待信号WAIT
    - 分离事务：总线空闲时释放，提高利用率
  - 5 常见总线标准
    - 系统内：ISA、PCI、PCIe（串行、高速）
    - 设备：USB（串行、热插拔）、SATA、SCSI
- 七、输入/输出系统 ★★★
  - 1 I/O接口
    - 功能：数据缓冲、电平匹配、地址译码、命令/状态寄存器
    - 编址：统一编址（存储器映射，可用访存指令）/ 独立编址（专用IN/OUT指令）
    - 【易错】独立编址不占主存地址空间，但需专门指令
  - 2 I/O方式对比 ★★★
    - 程序查询：CPU串行等待，接口最简单，效率最低
    - 程序中断：CPU与I/O并行；需中断机构
    - DMA：主存与I/O间直接成块传送，CPU仅在开始/结束介入
      - 【易错】DMA请求优先级高于中断请求（避免数据丢失）
    - 【易错】三种方式数据流向：中断需经CPU寄存器中转；DMA不经过CPU
  - 3 中断处理流程 ★★★
    - 完整流程
      - 关中断 → 保存断点（PC） → 引出中断服务程序（向量地址） → 保存现场 → 开中断 → 执行服务 → 关中断 → 恢复现场 → 开中断 → 中断返回
    - 中断向量：中断服务程序入口地址；向量地址：存放入口地址的地址
    - 多重中断（嵌套）
      - 屏蔽字：屏蔽字某位为1表示屏蔽该中断源
      - 【易错】中断处理优先级由屏蔽字决定（响应优先级由排队器硬件决定）
      - 【考点】屏蔽字设置：高优先级中断服务程序屏蔽自身及以下，开放更高级
  - 4 DMA ★★
    - 传送流程
      - 预处理（初始化） → 数据传送（完全硬件） → 后处理（中断CPU）
    - 传送方式：停止CPU访问 / 周期挪用（窃取） / 交替访问
    - 【易错】周期挪用挪的是存储周期，CPU与DMA交替访存
- 八、组成原理易错陷阱30条（考前冲刺）
  - 1 冯·诺依曼机核心是存储程序，以存储器为中心（现代）
  - 2 CPU执行时间 = 指令数×CPI×时钟周期，主频只是一环
  - 3 补码0唯一；8位补码范围−128~+127，−128无原码
  - 4 由[X]补求[−X]补：连同符号位取反加1
  - 5 判断溢出：符号位进位⊕最高数值位进位；或双符号位01/10
  - 6 补码加减符号位一起运算；尾数溢出≠浮点溢出（可右规）
  - 7 IEEE754：单精度偏置127，尾数隐含1；阶码全0为非规格化数
  - 8 浮点对阶：小阶向大阶，小阶尾数右移
  - 9 浮点加减顺序：对阶→尾数加减→规格化→舍入→判溢出
  - 10 海明码 $2^r \ge k+r+1$，校验位放2^i位
  - 11 SRAM快不刷新做Cache；DRAM慢需刷新做主存，刷新按行
  - 12 DRAM地址分行列两次送（RAS/CAS），引脚减半
  - 13 低位交叉存储体流水启动间隔 T/m
  - 14 Cache映射：直接=块号mod行数；组相联组号=块号mod组数
  - 15 直接映射不需要替换算法
  - 16 写分配法配回写法；非写分配配全写法
  - 17 平均访问时间公式：h×Tc+(1−h)(Tc+Tm)，别漏"未命中也先访Cache"
  - 18 LRU用栈实现；Cache对程序员透明
  - 19 相对寻址EA=新PC+偏移（PC已+1）
  - 20 基址面向系统（重定位）；变址面向用户（数组）
  - 21 扩展操作码：全1编码留给后续扩展
  - 22 PC用户不可见；PSW用户可见（条件码）
  - 23 微程序控制器存于控制存储器（ROM）
  - 24 流水线冒险：结构（资源冲突）、数据（RAW）、控制（转移）
  - 25 load-use数据冒险必须暂停1周期（转发不能完全消除）
  - 26 系统调用属于内异常（陷入）；DMA结束是外中断
  - 27 异步定时握手适合速度差异大的设备
  - 28 中断响应优先级（硬件排队）与处理优先级（屏蔽字）可以不同
  - 29 DMA传送期间CPU不干预；DMA优先级高于中断请求
  - 30 周期挪用：DMA与CPU交替占用总线，CPU效率损失小
- 九、大题标准话术
  - Cache计算：先求主存地址划分（标记+组号+块内地址），按映射方式定Cache行号，计算命中率后代入平均访问时间公式
  - 浮点运算：按IEEE754展开阶码（减偏置）与尾数（加隐含1），按对阶→加减→规格化→舍入→判溢出五步书写
  - 流水线：画出时空图，完成时间=(k+n−1)周期，吞吐率=n/(k+n−1)，加速比=顺序时间/流水时间
  - 中断嵌套：给出各中断源屏蔽字表，按"1屏蔽0开放"分析执行轨迹，画出CPU运行轨迹图
  - 指令执行：取指→间址→执行→中断四周期分析数据流（PC→MAR→MDR→IR等微操作序列）

Presented with xmind